Dossier de projet — Agent IA support & cybersécurité auto-apprenant

19 octobre 2026 · Antoine Riom et Florent · 5e année, domaine professionnel IA

> **Projet de 5e année IA — binôme Antoine Riom et Florent — 9 semaines réparties sur l'année.** Nous construisons un agent IA qui traite seul les tickets support et cybersécurité simples, escalade les autres, et apprend des réponses validées des humains pour en traiter toujours plus.

1. Contexte et problématique

Entreprise cible (hypothèse de cadrage) : ETI de services d'environ 2 000 salariés, avec un service desk interne de niveau 1-2 et un petit SOC. Les chiffres ci-dessous sont des hypothèses de dimensionnement, à confirmer sur données réelles.

- **Support** : environ 2 000 tickets par mois, dont une large part répétitive (mot de passe, VPN, accès, imprimantes). Chaque ticket simple mobilise un technicien 10 à 20 minutes, et la réponse dépend de qui le traite.
- **Cybersécurité** : le SIEM et l'IDS produisent beaucoup d'alertes, majoritairement de faible gravité ou des faux positifs. Les analystes saturés trient tard, ce qui allonge le temps de réponse aux vrais incidents, dont le coût croît avec la durée d'exposition.
- **Perte de savoir** : les solutions trouvées par les techniciens restent dans les tickets clos et ne sont jamais capitalisées.

Problématique : comment automatiser de façon fiable et traçable le traitement des tickets simples, support comme cyber, tout en faisant progresser le système à partir du travail humain, sans lui donner un pouvoir d'action dangereux ?

2. Objectifs et indicateurs

L'agent doit (1) classer et prioriser chaque ticket, (2) répondre seul aux tickets support simples avec sources, (3) analyser et diagnostiquer les tickets cyber et exécuter les remédiations simples autorisées, (4) escalader le reste avec un résumé exploitable, (5) intégrer au graphe les résolutions humaines validées.

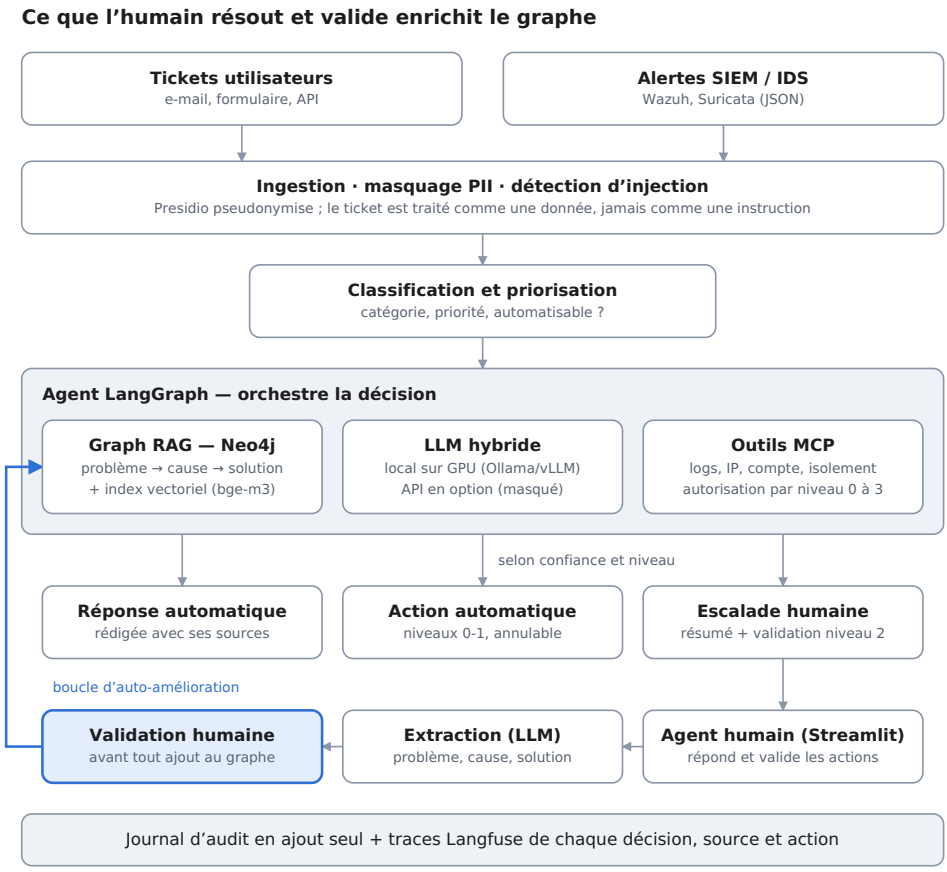
Indicateur	Définition	Cible fin de projet
Taux de résolution automatique	Tickets clos sans humain / total	≥ 30 % support, ≥ 20 % cyber
Précision des réponses	Réponses jugées correctes et sourcées sur le jeu annoté	≥ 90 %
Précision de classification	Catégorie et priorité justes (F1 macro)	≥ 0,85
Taux d'escalade justifiée	Escalades nécessaires / escalades totales	≥ 80 %
Actions dangereuses	Action exécutée hors autorisation	0
Temps moyen de traitement	Ticket reçu → réponse ou escalade	< 1 min
Gain d'apprentissage	Hausse du taux d'auto-résolution après 2 cycles d'enrichissement du graphe	+ 10 points

3. Périmètre

Dans le périmètre	Hors périmètre
Tickets texte en français et anglais (e-mail, formulaire, API)	Tickets vocaux, pièces jointes complexes (captures, PDF)
Alertes SIEM/IDS simulées au format JSON (type Wazuh, Suricata)	Connexion à un SOC ou un AD de production
Outils de remédiation sur un environnement de laboratoire (pare-feu, annuaire, EDR simulés)	Forensique avancée, réponse à incident majeur
Graph RAG + validation humaine des connaissances	Ré-entraînement (fine-tuning) du LLM
Interface web agent / validateur, démo locale	Mise en production, haute disponibilité, multi-entreprise

4. Architecture technique

Chaque ticket suit le même flux : il est masqué, classé, puis l'agent décide de répondre, d'agir ou d'escalader ; la réponse finale de l'humain, une fois validée, enrichit le graphe.



Architecture · flux d'un ticket et boucle d'auto-amélioration

- **Graph RAG** : le graphe relie *Problème*, *Symptôme*, *Cause*, *Solution*, *Actif* et *Ticket* ; la recherche combine similarité vectorielle et parcours des relations (ex. même cause déjà vue sur un autre service).
- **Décision** : l'agent n'agit seul que si la confiance dépasse un seuil, qu'une source existe et que l'action est autorisée à son niveau ; sinon il escalade avec son analyse, son diagnostic et la priorité proposée.
- **Auto-amélioration** : le LLM extrait de la réponse humaine les triplets problème → cause → solution ; un validateur les accepte, corrige ou rejette avant l'écriture dans Neo4j.

5. Technologies et justification

Brique	Choix	Justification
LLM	**Hybride** : modèle open-weights 7-14B (famille Qwen ou Mistral) servi par Ollama/vLLM sur GPU local ; API commerciale en option	Les tickets cyber et les données personnelles restent en local ; l'API sert de référence de qualité sur des tickets anonymisés
Embeddings	Modèle multilingue local (bge-m3)	Français et anglais, exécution locale, gratuit
Graphe + vecteurs	**Neo4j** (graphe + index vectoriel natif)	Une seule base pour les relations problème → cause → solution et la recherche sémantique ; requêtes Cypher lisibles
Orchestration d'agents	**LangGraph**	Machine à états explicite, points d'arrêt pour validation humaine, reprise sur erreur
Outils de l'agent	Serveurs **MCP** (bloquer IP, réinitialiser compte, isoler hôte, chercher dans les logs)	Interface standard, chaque outil isolé avec ses propres droits
Masquage PII	Microsoft **Presidio**	Open source, détecteurs personnalisables (e-mails, IP internes, noms)
API	**FastAPI** + Docker Compose	Typage, documentation OpenAPI automatique, déploiement reproductible
Interface	**Streamlit**	Rapide à développer pour la file de tickets et l'écran de validation
Observabilité et évaluation	**Langfuse** (auto-hébergé) + **Ragas** + pytest	Traces de chaque décision, métriques RAG standard, tests automatisés

6. Sécurité et garde-fous

- **Données personnelles** : masquage Presidio dès l'ingestion (pseudonymes réversibles, table de correspondance chiffrée côté serveur) ; aucune donnée non masquée n'est envoyée à l'API externe.
- **Injection de prompt** : le contenu du ticket est traité comme une donnée, jamais comme une instruction (balisage dédié dans le prompt) ; détecteur d'injection en amont ; l'agent ne peut pas modifier ses propres droits ; les paramètres d'action (IP, compte) sont vérifiés par du code, pas par le LLM.
- **Niveaux d'autorisation des actions** :

Niveau	Exemples	Règle
0 — Lecture	Chercher dans les logs, consulter un compte	Automatique
1 — Réversible, faible impact	Bloquer une IP externe 24 h, forcer un changement de mot de passe	Automatique si confiance ≥ seuil et cible hors liste protégée
2 — Impact métier	Isoler une machine, désactiver un compte	Validation humaine obligatoire
3 — Interdit	Supprimer des données, modifier les règles de sécurité globales	Jamais par l'agent

- **Journalisation** : chaque décision, source utilisée et action est tracée (Langfuse + journal d'audit en ajout seul), avec bouton d'annulation pour le niveau 1.
- **Validation du graphe** : aucune connaissance n'entre dans Neo4j sans validation d'un humain, ce qui évite d'empoisonner la base avec une mauvaise réponse ou un ticket malveillant.

7. Données et évaluation

- **Base de connaissance** : une vingtaine de procédures IT rédigées pour l'entreprise fictive (VPN, MFA, messagerie, postes) et des fiches de réponse cyber inspirées de MITRE ATT&CK.
- **Tickets support** : jeux publics (ex. *Customer Support Ticket Dataset* sur Kaggle) complétés par des tickets synthétiques générés par LLM puis relus.
- **Tickets cyber** : alertes produites à partir de CIC-IDS2017 ou UNSW-NB15 et de scénarios rejoués en laboratoire (force brute SSH, scan de ports, phishing signalé).
- **Jeu annoté de test** : environ 300 tickets (200 support, 100 cyber) annotés à deux : catégorie, priorité, « automatisable ou non », réponse ou action attendue ; dont 30 tickets piégés (injection de prompt, demande d'action interdite).
- **Protocole** : jeu tenu à l'écart du développement ; mesure des indicateurs de la section 2 ; fidélité et pertinence des sources avec Ragas ; comparaison RAG vectoriel seul / Graph RAG et modèle local / API ; simulation de la boucle d'apprentissage en 2 cycles (les réponses humaines du cycle 1 enrichissent le graphe avant le cycle 2).
- **Tests** : tests unitaires des outils et des règles d'autorisation, tests d'intégration du pipeline, suite de non-régression rejouée à chaque modification du prompt ou du modèle.

8. Livrables et planning

Livrables : code source (dépôt Git, Docker Compose, CI de tests) ; démo vidéo de 5 minutes ; jeu de données de test annoté ; rapport final avec résultats chiffrés ; documentation technique et guide d'installation ; support de présentation.

Les 9 semaines projet sont réparties dans l'année entre les périodes de cours ; chaque semaine se termine par un point d'avancement et une version fonctionnelle. **A** = Antoine (IA, RAG, graphe), **B** = Florent (cybersécurité, outils, interface).

Semaine	Antoine (A)	Florent (B)	Jalon
S1	Cadrage, choix et benchmark rapide des LLM locaux	Environnement Docker, labo cyber (pare-feu, annuaire, EDR simulés)	Cahier des charges validé
S2	Base de connaissance, ingestion, RAG vectoriel	Génération des tickets et alertes, début d'annotation	Premier RAG qui répond avec sources
S3	Classification et priorisation	Masquage PII, API FastAPI	Pipeline ticket → catégorie
S4	Modèle de graphe Neo4j, Graph RAG	Serveurs MCP des outils de niveau 0 et 1	Graph RAG opérationnel
S5	Orchestration LangGraph (répondre / agir / escalader)	Niveaux d'autorisation, journal d'audit, annulation	Agent de bout en bout
S6	Boucle d'auto-amélioration (extraction → validation → graphe)	Interface Streamlit (file, validation, actions niveau 2)	MVP complet
S7	Évaluation complète, comparaisons local/API et RAG/Graph RAG	Tests de sécurité (injection, actions interdites), correctifs	Résultats chiffrés
S8	Optimisation des prompts et des seuils	Durcissement, documentation	Version gelée
S9	Rapport, résultats	Démo vidéo, présentation	Rendu final

9. Risques du projet et évolutions

Risque	Impact	Parade
Modèle local trop faible pour le raisonnement cyber	Résolution automatique basse	Comparaison avec l'API dès S1 ; seuils de confiance stricts, escalade par défaut
Hallucination ou réponse non sourcée	Perte de confiance	Réponse refusée sans source ; vérification de fidélité avant envoi
Données de test peu réalistes	Résultats peu crédibles	Mélange jeux publics et synthétiques relus, scénarios rejoués en labo
Semaines projet éclatées dans l'année	Perte de contexte, retard	Version fonctionnelle chaque semaine, dépôt et documentation tenus à jour
Périmètre trop large	MVP incomplet	Priorité au support en S2-S4, cyber limité aux 3 scénarios du labo si retard

Pistes d'évolution : connecteurs vers de vrais outils (Jira Service Management, GLPI, Wazuh, Active Directory) ; prise en charge des captures d'écran ; fine-tuning du modèle local sur les tickets validés ; détection des incidents récurrents par analyse du graphe ; déploiement multi-équipes avec droits par périmètre.